

Implementing a stack with a linked list is slightly slower than with an array because memory allocation is necessary to create a new link for each insertion, instead of allocating all the space needed at the beginning. Deallocation of the links is necessary at some point following removal of an item from the list. This might be done explicitly by the program or through a process known as garbage collection (both of which take time). The linked list implementation can either be more or less memory efficient than the array implementation. Linked lists require storage of the pointers to the next link, while arrays can have many unused cells if the stack size is much smaller than what was allocated.

Queue

Use a queue when you want access only to the first data item inserted; it's a first-in, first-out (FIFO) structure.

Like stacks, queues can be implemented as arrays or linked lists. Both are efficient with $O(1)$ insertion and deletion. The array requires additional programming to handle the situation in which the queue wraps around at the end of the circular array. A linked list must be double-ended to allow insertions at one end and deletions at the other. Both array and double-ended linked list implementations also allow $O(1)$ access to peek at the last item inserted but not to delete it.

As with stacks, the choice between an array implementation and a linked list implementation is determined by how well the maximum number of items can be predicted. Use the array if you know about how much data there will be; otherwise, use a linked list.

Priority Queue

A priority queue is used when the only access desired is to the data item with the highest priority. This is the item with the largest (or sometimes the smallest) key.

Priority queues can be implemented as an ordered array or as a heap. Insertion into an ordered array is slow, but deletion is fast. With the heap implementations, both insertion and deletion take $O(\log N)$ time. The drawback of the heap implementation is that it does not preserve FIFO ordering among items with equal keys.

The heap implementation is almost always faster than implementing as an ordered array. Memory usage of these types varies but is always $O(N)$. If the maximum number of items can be predicted in advance, the sorted array needs the least memory. If the size is hard to predict, both ordered arrays and heaps can be grown as needed without degrading overall insertion time but at the expense of having more unused memory.

Use an ordered array if FIFO ordering of equal keyed items is required, and it's difficult to reliably prioritize items by insertion time. For all other priority queues, the heap is the best choice.

Comparison of Special-Ordering Structures

Table 16-2 shows the Big O times for insertion and deletion on stacks, queues, and priority queues and whether the items can be traversed according to the order of their keys in $O(N)$ time. These structures don't support fast searching.

Table 16-2 *Special-Ordering Data Structure Speeds*

Data Structure	Insertion	Deletion	Traversal in Key Order	Comment
Stack (array or linked list)	$O(1)$	$O(1)$	No	Deletes most recently inserted item (LIFO)
Queue (array or linked list)	$O(1)$	$O(1)$	No	Deletes least recently inserted item (FIFO)
Priority queue (ordered array)	$O(N)$	$O(1)$	Yes	Deletes highest-priority item
Priority queue (heap)	$O(\log N)$	$O(\log N)$	No	Deletes highest-priority item

Sorting

The widespread availability of libraries with good sorting capabilities has made writing sorting methods rare. As a first step, look for well-tested libraries and

try them on the problem data. If you can't find one that performs well enough, you may need to implement your own.

For data that will fit in memory, it's worthwhile initially to try a slow but simple sort, such as the insertion sort. Your computing platform might have enough processing speed to sort your data in a reasonable time. Remember to try it with many different initial orderings of the data because the performance of insertion sort (and other algorithms) can vary significantly with the input ordering.

Insertion sort is the best of the simple sorting algorithms we saw in [Chapter 3](#), “[Simple Sorting](#),” and is especially good for almost sorted data, operating in about $O(N)$ time if not too many items are out of place. For example, when adding 10 items to an already-sorted file of 20 million, insertion sort should be very fast.

If the insertion sort proves too slow, then it's likely you'll need an $O(N \times \log N)$ algorithm, like those we introduced in [Chapter 7](#), “[Advanced Sorting](#).” These can be significantly more complex, so having a simple but slow algorithm like insertion sort is good for testing the correctness of your next method.

You could try the Shellsort next. It's fairly straightforward to implement and should get the performance to $O(N^{3/2})$. You need to implement an interval sequence, and Knuth's is likely to be fine.

If the Shellsort proves too slow, you should use one of the more complex but faster sorts: mergesort, heapsort, or quicksort. Mergesort requires extra memory and is the best choice if the data will not fit in memory. When there's not enough RAM, the merging can be done by reading items one at a time from two sorted files and outputting the lowest item to a third file. The file merging is often paired with an in-memory sorting method like Shellsort or heapsort to sort small sections of the initial file data and write them to disk

For in-memory sorting, the heapsort described in [Chapter 13](#), “[Heaps](#),” has distinct advantages. The data can be sorted within an initial array without requiring extra memory. It sorts the data in $O(N \times \log N)$ time regardless of the initial data ordering. A side benefit is that the heapify method used to make the initial array into a heap can be used for making heaps in $O(N)$ time, enabling calculations like order statistics without a full sort. The disadvantage of heapsort is its somewhat more complicated algorithm, which ends up making more comparisons than quicksort, although both are $O(N \times \log N)$ overall.

If complex algorithms are not a deterrent, Timsort is also an option. While the average performance of Timsort is also $O(N \times \log N)$, it has the added benefit of running in $O(N)$ time for initially sorted data.

[Table 16-3](#) summarizes the running time for the sorting algorithms. The column labeled Algorithm Complexity attempts to categorize the level of difficulty in understanding and correctly implementing the algorithm. This, of course, depends greatly on the programmers doing the implementation. What's simple for some people can be complex for others.

Table 16-3 *Comparison of Sorting Algorithm Speeds*

Sort	Best	Average	Worst	Algorithm Complexity	Extra Memory
Bubble	$O(N^2)$	$O(N^2)$	$O(N^2)$	Simple	No
Selection	$O(N^2)$	$O(N^2)$	$O(N^2)$	Simple	No
Insertion	$O(N)$	$O(N^2)$	$O(N^2)$	Simple	No
Shellsort	$O(N^{3/2})$	$O(N^{3/2})$	$O(N^{3/2})$	Moderate	No
Quicksort	$O(N \times \log N)$	$O(N \times \log N)$	$O(N^2)$	Difficult	No
Mergesort	$O(N \times \log N)$	$O(N \times \log N)$	$O(N \times \log N)$	Moderate	Yes
Heapsort	$O(N \times \log N)$	$O(N \times \log N)$	$O(N \times \log N)$	Difficult	No
Timsort	$O(N)$	$O(N \times \log N)$	$O(N \times \log N)$	High	Yes

Specialty Data Structures

Many data structures are created to address particular problem areas. They work well for their designed goals but can't really serve as general-purpose data stores. Although they may support insertion, deletion, search, and traversal of items, they differ in their primary operations of interest. You can turn to these structures based on what operations are needed.

Quadrees and Grids

When the data items have a spatial component, located in either a two-dimensional space such as a map or in three dimensions, you need structures that enable fast searches of those spaces. Quadtrees and grids offer a way of capturing the spatial information of points or regions so that items can be found by position in a two-dimensional space. Octrees offer a similar structure for items in three-dimensional space.

By organizing the items by their coordinates, quadtrees enable fast searches for the items nearest to or the items within a certain radius of a given point. Similar to binary trees, these kinds of searches (plus insertion and deletion) can be done in $O(\log N)$ time. This is much faster than a simple list of points or grids when there are large numbers of points that cluster in a few regions.

Grids can perform very well for smaller collections of points. They can outperform quadtrees when the points are close to a uniform distribution within known bounds.

Graphs

Graphs can be used to model real-world situations. The structure of the graph reflects the structure of the problem such as in transportation and communication networks. They are also used in other algorithms such as the PageRank algorithm—the initial ranking mechanism of Google’s search results. The interlinked structure of websites can be modeled with a directed graph.

When you need a graph, nothing else will do, so there’s no decision to be made about when to use one. The primary choice is how to represent the graph: using an adjacency matrix or adjacency lists. Your choice depends on whether the graph is dense, when the adjacency matrix is preferred, or sparse, when the adjacency list or adjacency hash table should be used.

The special operations that graphs handle include finding paths, trees, and subgraphs. The depth-first search and breadth-first search for a particular vertex or kind of vertex run in $O(V^2)$ time, where V is the number of vertices, for adjacency matrix representation. They run in $O(V+E)$ time, where E is the number of edges, for adjacency list representation. Finding minimum spanning trees and shortest paths run in $O(V^2)$ time using an adjacency matrix and $O((E+V) \times \log V)$ time using adjacency lists. The number of edges can be on the order of V^2 in dense graphs, so you need to estimate V and E for your graph and do the arithmetic to see which representation is appropriate.

External Storage

For most of the data storage structures, we assumed that data was kept in main memory. When the data is too large to fit in memory, some or all of it must be stored in external storage, which often means disk files. We discussed external storage in the second parts of [Chapter 9](#), “[2-3-4 Trees and External Storage](#),” and [Chapter 11](#), “[Hash Tables](#).”

When data is stored in a disk file, we assumed that it was organized in fixed-size units called blocks, each of which holds a certain number of records. A record in a disk file holds the same sort of data as an object in main memory. Like an object, a record has at least one key value used to access it.

We also assumed that reading and writing operations always involve a single block, and these read and write operations are far more time-consuming than any processing of data in main memory. For fast operation, the number of disk accesses must be minimized.

Sequential Storage

The simplest approach to external storage is to store records randomly and read them sequentially when searching for one with a particular key. New records can simply be inserted at the end of the file. Deleted records can be marked as deleted, or records can be shifted down (as in an array) to fill in the gap.

On the average, searching and deletion involve reading half the blocks, so sequential storage is not very fast, operating in $O(N)$ time, where N is the number of blocks. Still, it might be satisfactory for a small number of records.

Indexed Files

Speed increases dramatically when indexed files are used. In this scheme an index of keys and corresponding block numbers is kept in main memory. To access a record with a specified key, the index is consulted. It supplies the block number for the key, and only one block needs to be read, taking $O(1)$ time.

Several indices with different kinds of keys can be used (one for last names, one for telephone numbers, and so on). This scheme works well until the index

becomes too large to fit in memory. When that is the situation, the index files are themselves stored on disk and read into memory as needed.

The disadvantage of indexed files over sequential storage is that the index must be created and maintained. This process involves reading through all the records sequentially, so creating the index could be slow, if it's created after many external records have been stored. Also, the index needs to be updated when items are inserted into and deleted from the file.

B-trees

B-trees are multiway trees, commonly used in external storage, in which nodes correspond to blocks on the disk. As in other trees, the algorithms find their way down the tree, reading one block at each level. B-trees provide searching, insertion, and deletion of records in $O(\log N)$ time. This is quite fast and works even for very large files. The programming, however, is not trivial.

Hashing

If it's acceptable to use about twice as much external storage as a file would normally take, then external hashing might be a good choice. It has the same access time as indexed files, $O(1)$, but can handle larger files.

Choosing Among External Storage Types

The choice of what scheme to use in managing external storage depends on the problem's characteristics. If multiple indices to the records must be maintained (for example, by name, by telephone, by address), using indexed files is really the only option. This option takes a bit longer for insertions and deletions because multiple indices must be updated for each operation, but it provides the fastest access for searches that use one or more of the indices.

If only a single index is needed, then B-trees or hashing become attractive. B-trees use less disk space, and hashing provides the fastest search access.

Virtual Memory

Sometimes you can let your operating system's virtual memory capabilities solve disk access problems with little programming effort on your part.

If you skipped the complications of B-trees or hashing and simply used sequential storage, you can read large disk files into an array. If that array is too big to fit in main (RAM) memory, the virtual memory system keeps part of that array in main memory and stores the rest on the disk. As you access different parts of the array, they are read from the disk automatically and placed in memory. This is typically done in chunks of memory called *pages*.

You can apply internal algorithms to the entire file contents by assuming the corresponding array was in memory at the same time, and let the operating system worry about reading the appropriate part of the file if it isn't in memory already. Operating systems have some sophisticated methods of predicting what pages of memory will be needed next and ensuring they are loaded quickly.

Of course, the operation on such a huge array is slower than when the entire array is in memory, but changing your algorithm to use B-trees or hashing to find records also takes time. It's unclear which will be faster, and it is quite possible for virtual memory paging to be faster or slower than one of the external-storage algorithms depending on the amount of data. Given that uncertainty, it may be worth simply ignoring the fact that a file doesn't fit in memory and seeing how well your algorithms work with the help of virtual memory. If the simple implementation doesn't perform well enough, then you can invest time in optimizing the external memory accesses.

Onward

We've come to the end of our survey of data structures and algorithms. The subject is large and complex, and no one book can make you an expert. We hope this book has given you a taste for them and taught you the fundamentals. If that taste intrigued you, many, many more data structures and variations on the ones described here await you. Some researchers spend their entire careers inventing, developing, and analyzing their performance. Those of us who simply use them as building blocks benefit by interpreting those analyses and comparing them to the requirements of our jobs.

[Appendix B](#), “[Further Reading](#),” contains suggestions for further study.

Appendix A. Running the Visualizations

In This Appendix

- [For Developers: Running and Changing the Visualizations](#)
- [For Managers: Downloading and Running the Visualizations](#)
- [For Others: Viewing the Visualizations on the Internet](#)
- [Using the Visualizations](#)

We believe that active learning is the best way to master new material. Reading a book and attending a lecture provide the key concepts, but applying that knowledge by solving practice problems and performing experiments makes the abstract ideas more concrete.

To understand data structures and algorithms, it helps to see them in action. To that end, we have developed interactive visualizations of most of the data structures and algorithms described in this book. These programs show the step-by-step processing of data in each structure, allowing you to see all the key operations. You can try the structures with your own data to see how they handle it. The individual chapters suggest exercises for you to try with the visualizations to ensure that you see the important behaviors of each structure and algorithm.

There are several ways for you to run the visualization programs. The best way for each reader depends on how much programming you plan to do. For readers who plan to or already work as software developers, we encourage you to download the source code and run the programs on a computer with an editor or other integrated development environment. You can even help future students by contributing to the open-source software project for the visualizations. Your experience can help suggest new ways of making the

complex ideas easier to understand by others. Follow the instructions in “For Developers: Running and Changing the Visualizations.”

If you don’t plan to do much programming but still want to try running the visualizations on your own data, you can download them and run them on a personal computer. This approach would be a good choice for someone who manages or analyzes software development, without doing much coding on their own. If that suits your needs, follow the instructions in “For Managers: Downloading and Running the Visualizations.”

If you don’t plan on doing any programming and have a connection to the Internet, you can run the visualizations in a web browser. The programs were written in Python and display their visualization using Tk, which means they don’t execute in a browser. That adds extra layers of interface that somewhat limits what you can do and see. For someone without access to a computer for software development, however, this may be the only option. If that’s the case, follow the instructions in “For Others: Viewing the Visualizations on the Internet.”

After you have been able to launch the visualizations, make sure to read the section “Using the Visualizations” at the end. This section explains some of the key features of all the visualizations and their user interfaces, including some limitations in the visualization that are not necessarily present in actual implementations of the data structures.

For Developers: Running and Changing the Visualizations

To maximize your ability to run the programs and modify them for your own experiments or to contribute to the open-source software project, you need at least three pieces of software on the computer where you plan to do the development:

- Python
- Git
- The visualization software

The first two come in different versions depending on your computer’s operating system.

Getting Python

To get the Python programming language, you have many options. First, check whether it is already available on your system. If you're already familiar with command-line interfaces like the Windows Power Shell, the Windows Command interpreter, or macOS and Linux Terminals, try launching one of those interfaces and typing the command:

```
python3
```

If Python release 3 is installed, this should produce some lines that look something like:

```
Python 3.7.5 (default, Nov  4 2019, 10:26:32)
[Clang 9.0.0 (clang-900.0.39.2)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

If this code doesn't show up, then you probably need to install Python. There are many places on the Internet to get Python. We explain only the primary one: www.python.org/downloads. This website has all the releases of Python and many preconfigured downloads for the different operating systems. It's also the official source for all information about the programming language, the different releases, documentation, and proposed changes for the future. The visualizations were made using Python release 3. You need a version that starts with "3."; do not use one that starts with "2." or lower. If this is your first time using Python, you may want to follow the documentation links to read the guides for beginners.

Getting Git

The second tool, git, manages collections of software and their different versions. It's exceptionally well suited to managing source code because it provides many ways to track the changes in text files. Git became the tool of choice for the developers of the Linux operating system. One of its key features is the distributed nature of the software collections, called *repositories*, that it manages. While github.com maintains a copy of most of these repositories, every developer gets their own complete repository including all the history of changes, in their local copy of the repository.

The `git` command-line tool is incredibly powerful, and not surprisingly, incredibly complex. It was invented by software professionals for use by software professionals. Its complexity, however, can be a hindrance for newcomers. Many users just need the basics, and graphical user interfaces can make that much easier. The GitHub Desktop tool provides a straightforward graphical interface to working with repositories, their branches, and commits (changes) to those branches.

Learning how to use the `git` command-line tool is beyond the scope of this book. Instead, we describe how to use GitHub Desktop. If you're already familiar with the `git` command-line tool, however, go ahead and use that, because it can do everything GitHub Desktop does and more.

Use your web browser to visit <https://desktop.github.com>. It should auto-detect the operating system of your computer and propose a download option. At the time of this writing, only Windows and macOS are supported (on Linux, you need to use the `git` command-line tool, which is often bundled with the Linux distribution). Follow the download instructions to install GitHub Desktop.

Both GitHub Desktop and the `git` command-line tool allow you to review the history of changes in a repository and make new changes to the files it contains. In the next section, we discuss how to get the repository that contains the visualizations for this book. If you plan to make contributions to the open-source visualization project, you also need an account on the GitHub.com website. That is where the project's repository is stored, and your user account allows you to contribute to many different projects. Go to <https://github.com> and select the Sign up button to create a new account.

Getting the Visualizations

The third piece of software you need is the visualization code itself, which is bundled in a `git` repository. You can find that repository at <https://github.com/JMCanning78/datastructures-visualization>. The process of downloading a `git` repository is called *cloning* the repository. If you are using Github Desktop, go to the File > Clone Repository dialog. Choose the GitHub.com tab and then start typing in the repository name, `JMCanning78/datastructures-visualization`. The repository should pop up as a choice that you can select. The dialog lets you specify a Local Path on your computer where it will store the clone. After selecting the repository on

GitHub.com and specifying the local path, select the Clone button to make the clone.

If you're using the command-line version of `git`, the best way to get the visualization software is to put one of your command-line windows into the parent directory where you want to store the software and then run the `git clone` command. For example, if you wanted to store the repository in `/Users/Me/myrepositories`, the commands would look something like this:

```
cd/Users/Me/myrepositories
git clone https://github.com/JMCanning78/datastructures-visualization
```

The `cd` command is short for change directory. Your command-line tool might have a different name for the change directory command. The `git clone` command creates a subdirectory called `datastructures-visualization` that contains a clone of the git repository. You can use the `cd` command again to change the current working directory to be that subdirectory. From that directory, you can launch the visualizations by running

```
python3 DatastructureVisualizations.py
```

If you get a message about missing required modules, you may need to add some modules to your Python environment. Try running the Python installer program (`pip3`), as follows:

```
pip3 install -r requirements.txt
```

When that runs successfully, try the previous command to run the visualizations.

If you prefer to use an Integrated Development Environment (IDE) for your Python development work, you need to register the cloned repository with the IDE. From there, you should be able to run `DatastructureVisualizations.py` or any other Python programs.

For Managers: Downloading and Running the Visualizations

If you have a macOS or Windows desktop computer, you may be able to download a prepackaged collection of all the visualizations. It runs basically the same program as the `DatastructureVisualizations.py` described previously. Browse <https://datastructures.live/> and look for the links related to downloads.

The software bundles are cryptographically signed to help ensure the integrity of the software. Your computer and its malware protection software may ask whether you trust the downloaded visualization software before allowing you to run it.

For Others: Viewing the Visualizations on the Internet

You can view the visualizations in a web browser by visiting <https://datastructures.live/view/>. When you select the Run button, this site runs the visualizations on a server and copies the output to a pane in your web browser window. The initial screen looks something like [Figure A-1](#).

Visualizations from *Data Structures and Algorithms in Python*

Press the Run button below to see some data structure and algorithm visualizations for the upcoming textbook: *Data Structures and Algorithms in Python* by John Canning, Alan Broder, and Robert Lafore.

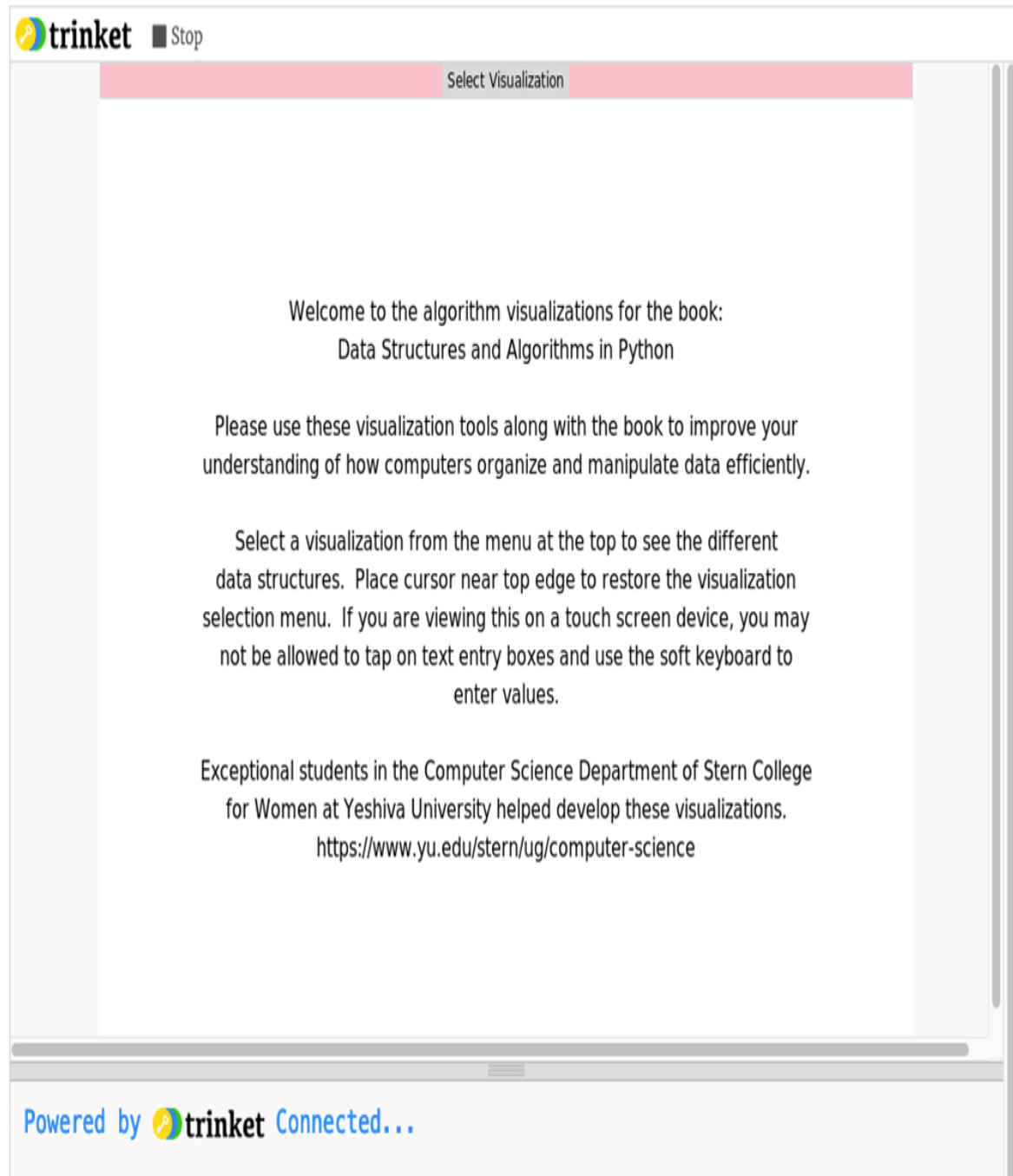


Figure A-1 Initial screen when viewing visualizations in a web browser

Select the Select Visualization button at the top to get a menu of all the visualizations that can be shown.

There are limitations on viewing the visualizations in a web browser. In particular:

- The window cannot be resized although you can increase the size of the upper pane by adjusting the size of the web browser window and dragging the separator bar just above the “Powered by trinket” text.
- The screen is being copied over the Internet, so there might be choppy or slow updates in response to interactions. Try using the visualization tool’s lower animation speed or stepping mode when this occurs.
- The viewing works best on computers with a mouse and keyboard. Viewing the trinket.io website on a device with only a touchscreen might not allow you to enter text for the arguments to operations. Selecting a text entry area does not put the keyboard focus on that area.

Using the Visualizations

The visualizations are small programs that usually show one data structure at a time. They all have a similar structure with a drawing area at the top and a set of controls on the bottom (see [Figure A-2](#)). There are specific instructions for how to use the visualizations in each of the chapters, and some general guidelines common to all of them follow.

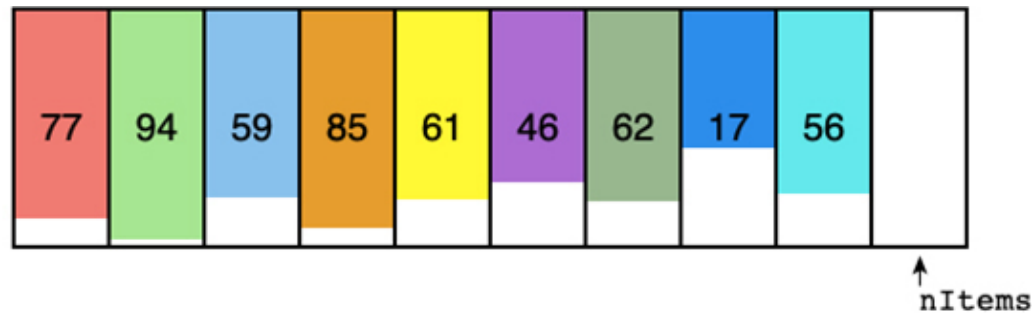


Figure A-2 *Drawing area at the top and a set of controls on the bottom*

The visualizations show the data in a drawing area at the top, such as in [Figure A-2](#). The Operations box in the lower left is where you initiate operations on the data structure. The buttons are grayed out and disabled when they are not applicable. In particular, the operations that take an argument like a key to insert are on the left side of the Operations box. They are grouped with a text entry area that must have a value entered before they become enabled. The hint in blue text explains what can be typed in the text entry area for the different operation arguments. Operations that take no arguments, like Traverse and Random Fill, are on the right.

Below the Operations box is a slider. This control can be adjusted to slow down or speed up the animation of the operations. During animations, the three small buttons on the lower right ▶ ▶▶ ■ are enabled to provide fine control. The leftmost of three buttons is either a triangle to resume playback, or two vertical bars ■▶■ to pause the animation, depending on the current state. The second button (with a triangle pointing into a vertical bar) steps forward in the

animation by one “step.” That button is enabled only when code is displayed because the steps are defined by the code. The square button stops the current operation and enables other operations to be run. Stopping an operation before it finishes can leave the data structure in an odd state, such as leaving extra copies of data items or not sorting all the items.

During most operations, a code box appears in the lower right (see [Figure A-3](#)). It shows the same program described in book, highlighting the part of the program being executed. In the figure, the items in an array are being sorted. The various local variables are shown along with the data structure in the drawing at the top. Only the three animation buttons are enabled while the sort operation runs. The highlighting on the single step button indicates that keyboard focus is on that button. Pressing the spacebar when a button has keyboard focus “presses” the button. Operations normally begin running in the “play” mode when a button is pressed. By holding down the Shift key when pressing the button, you can start the operation in stepping mode (if it displays code).

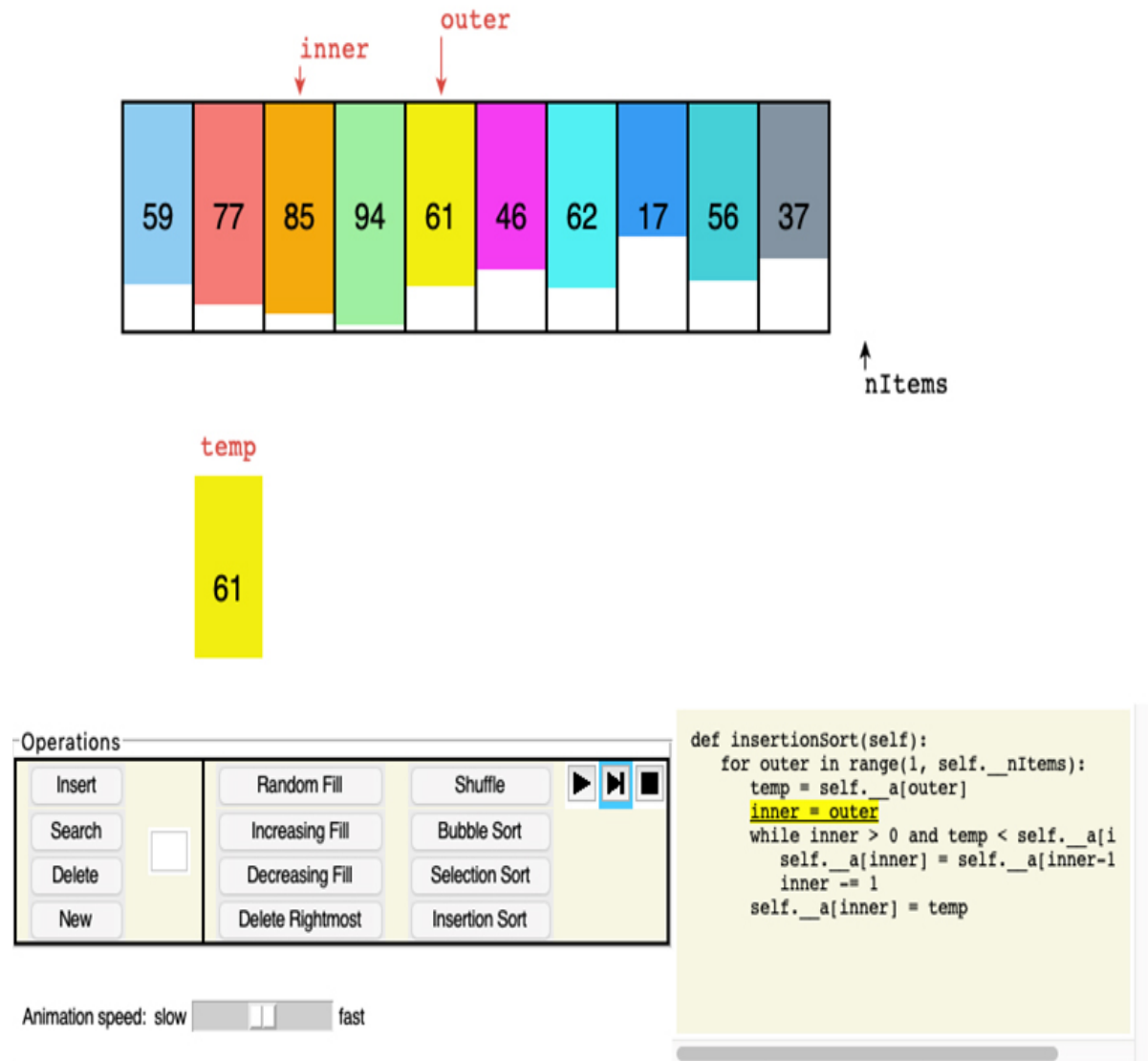


Figure A-3 *The code box in the lower right*

After each operation finishes, the code, and local variables remain on the display until the next operation is started. Status or error messages also appear at the bottom of the window, between the sliding speed control and the code box. The messages usually appear when operations finish. [Figure A-4](#) shows the display after the successful search for key 62 in the array.

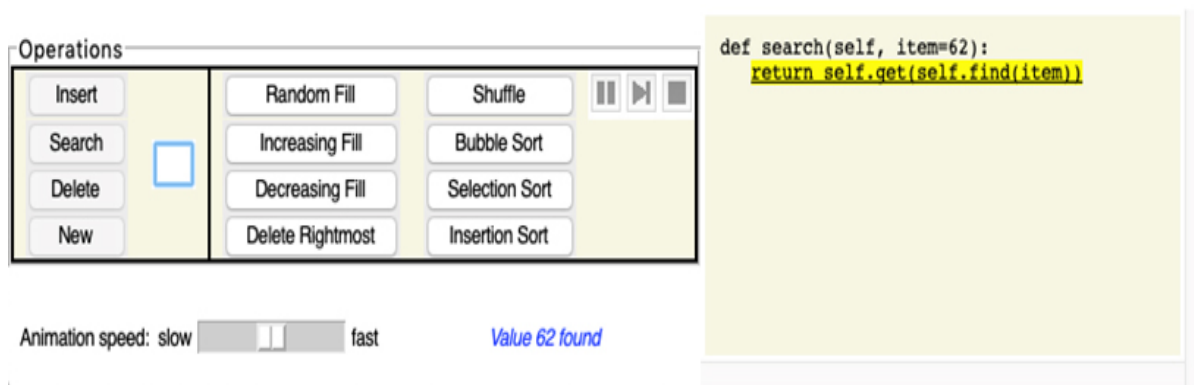
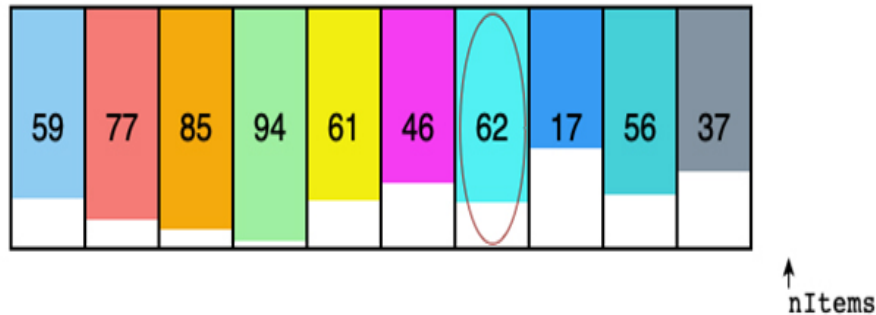


Figure A-4 Results of the search for key 62 in the array

In most visualizations, you can click a data item to select it and copy its value into the text entry area. This makes it less likely that a typographical error will prevent searching for or deleting an existing value.

Hint messages about the controls can appear next to the text entry box(es) and operations buttons. Hovering the mouse pointer over one of them displays a short hint about what values can be entered in that box or what the button does. When you first start one of the visualizations, it displays a message telling you all the kinds of things that can be entered in all the text entry box(es), like the message in [Figure A-2](#).

Depending on how you run the visualization, you may be able to resize the window. This capability is sometimes useful when the amount of data grows too large or the lines of code are wide. The code text box does not grow

vertically but can be scrolled for longer programs. Some visualizations require more display space and provide scrollbars to move around. At least one visualization also lets users zoom in and out on particular regions.

Appendix B. Further Reading

Because there are so many good books on computer science, data structures, and algorithms, it's hard to choose what's right to recommend. If we've sparked your interest to read further, then we've succeeded in one of our goals. There are many paths from which to choose. Here are a few that we've found to be especially good, organized into topic areas.

Data Structures and Algorithms

The definitive reference for any study of data structures and algorithms is *The Art of Computer Programming* by Donald E. Knuth, of Stanford University (Addison-Wesley, 2011). This seminal work was originally published in the 1970s and was cited as a major part of his Turing award in 1974. It currently consists of four volumes: *Volume 1: Fundamental Algorithms*, *Volume 2: Seminumerical Algorithms*, *Volume 3: Sorting and Searching*, and *Volume 4A: Combinatorial Algorithms, Part 1*. Of these, Volume 3 is the most relevant to the topics in this book. This work is highly mathematical and does not make for easy reading, but it is the bible for anyone contemplating serious research in the field. As of this writing, Prof. Knuth continues working on Volume 4B of the series releasing fascicles from time to time.

A somewhat more accessible text is Robert Sedgewick and Kevin Wayne's *Algorithms* (Addison-Wesley, 2016). This fourth edition of the book contains code examples written in Java and is adapted from earlier editions in which the code examples were written in Pascal, C, and C++. It is comprehensive and authoritative. The text and code examples are quite compact and require close reading.

Every computer scientist needs to have and read the encyclopedic *Introduction to Algorithms*, by Cormen, Leiserson, Rivest, and Stein (MIT

Press, 2009). The book provides clear, concise coverage of a startling range of topics, and is language agnostic—usefully providing algorithms in a generic pseudocode.

Programming Pearls by Jon Louis Bentley (second edition, Addison-Wesley, 2000) was originally written in 1986 but is nevertheless stuffed full of great advice for the programmer. Much of the material deals with data structures and algorithms.

We’ve covered high-level data structures and algorithms, but there’s a whole world of fascinating low-level bitwise data structures and algorithms that are still surprisingly useful in modern programming. For a great survey of such techniques, try *Hacker’s Delight* by Henry S. Warren Jr. (Addison-Wesley, 2013).

Recursion is a recurring theme throughout our book. Readers who are new to the concept and practice of writing recursive code can benefit by practicing recursively coding diverse problems. While these books address recursive solutions in Java, they are unbeatable sources of challenging problems: *Thinking Recursively with Java*, by Eric Roberts (Wiley, 2006), and *Practicing Recursion in Java*, by Irena Pevac (CreateSpace Independent Publishing, 2016).

Object-Oriented Programming Languages

For a clear, beautifully structured, step-by-step introduction to the concepts of object-oriented programming in Java, it is worth seeking out the now out-of-print *On to Java* by Patrick Henry Winston and Sundar Narasimhan (Pearson, 2002). One can only hope for a Python version of this classic book one day!

To get started in Python, try the comprehensive *Building Python Programs* by Reges, Stepp, and Obourn (Pearson, 2019). The book and its companion website provide extensive opportunities to methodically learn and hone your Python programming skills.

When you’re ready to master Python’s clever features, *Python Tricks—The Book*, by Dan Bader (Dan Bader, 2017) is a great resource. The book is filled with many self-contained nuggets of Python wisdom that you can

incrementally add to your repertoire, including how to best use Python's object-oriented features.

If you're already an experienced Python programmer, then *Python Distilled*, by David M. Beazley (Addison-Wesley Professional, 2021), should always be kept in reach on your desk for quick reference. This compact yet authoritative book contains almost everything you'll need to know about making the most effective and appropriate use of Python's most advanced features.

Object-Oriented Design (OOD) and Software Engineering

For an easy, nonacademic introduction to software engineering, try *The Object Primer: The Application Developer's Guide to Object-Orientation, Second Edition*, by Scott W. Ambler (Cambridge University Press, 2001). This short book explains in plain language how to design a large software application. The title is a bit of a misnomer; it goes way beyond mere OO concepts.

Object-Oriented Design in Java by Stephen Gilbert and Bill McCarty (Waite Group Press, 1998) is an unusually accessible text.

A classic in the field of OOD is *Object-Oriented Analysis and Design with Applications* by Grady Booch et al. (Addison-Wesley, 2007). The author is one of the pioneers in this field and the creator of the Booch notation for depicting class relationships. This book isn't easy for beginners but is essential for more advanced readers.

An early book on software design is *The Mythical Man-Month* by Frederick P. Brooks, Jr. (Addison-Wesley, 1996), which explains in a very clear and literate way some of the reasons why good software design is necessary. Since its original publication in 1975, it is said to have sold more copies than any other computer book.

Appendix C. Answers to Questions

Index

- [Chapter 1, “Overview”](#)
- [Chapter 2, “Arrays”](#)
- [Chapter 3, “Simple Sorting”](#)
- [Chapter 4, “Stacks and Queues”](#)
- [Chapter 5, “Linked Lists”](#)
- [Chapter 6, “Recursion”](#)
- [Chapter 7, “Advanced Sorting”](#)
- [Chapter 8, “Binary Trees”](#)
- [Chapter 9, “2-3-4 Trees and External Storage”](#)
- [Chapter 10, “AVL and Red-Black Trees”](#)
- [Chapter 11, “Hash Tables”](#)
- [Chapter 12, “Spatial Data Structures”](#)
- [Chapter 13, “Heaps”](#)
- [Chapter 14, “Graphs”](#)
- [Chapter 15, “Weighted Graphs”](#)

Chapter 1, “Overview”

1. insert, search, delete, traverse
2. c
3. If it is indented below a `class` definition with no intervening definition at the outer (unindented) level.
4. sorting
5. Constructors are functions that build an instance of an object class. Python uses the `__init__()` method of a class as the constructor.
6. You choose data structures that model the system and behavior being represented by the program. Among data structures that model the system equally well, you choose the ones that perform the most efficiently. Among those that perform equally fast, you choose ones that take the least amount of memory space.
7. A key field is used to find a larger record, typically by keeping a sorted structure of keys to accelerate the search.
8. It can reduce the amount of memory needed to store the data. It makes the program easier to understand, and hence, easier to maintain by other programmers.
9. To implement data structures and tightly organize algorithms with the data they manipulate.
10. b, d, & g

Chapter 2, “Arrays”

1. They should have `get()` and `set()` methods to access the value of a particular array cell. The ability to get and set cells indexed by an integer value is the basic functionality of arrays in all languages.
2. d. Some languages don't require specifying the maximum number of cells at construction time, but most do.
3. to prevent access to internal values of the object instance from being manipulated by code outside the object.
4. d

- 5. True
- 6. b
- 7. False
- 8. d
- 9. a
- 10. Both `find()` and `search()` return a numeric index to a cell in the ordered array containing an item being sought. If the item being sought is not in the array, `find()` returns the index of the next item just after the sought item, if any, while `search()` returns `None`. The versions in `OrderedRecordArray.py` search by key instead of the item itself.
- 11. raising to a power
- 12. 3
- 13. b
- 14. 6
- 15. False
- 16. a
- 17. constant
- 18. c
- 19. b
- 20. The number of records in the current, solar-system-based implementation is good for tens, maybe hundreds of records. For potentially billions of records that will need to be accessed in “random” order, a search method that is faster than $O(N)$ is needed. To get the $O(\log N)$ search speed, an ordered array is required, at a minimum. In fact, even more complex data structures are needed to get the search time down to something reasonable. The size of the individual records is not an important factor here.

Chapter 3, “Simple Sorting”

1. d
2. comparing and swapping (or copying/moving)
3. False
4. a
5. False
6. b
7. False
8. A swap would take three times longer than a copy operation because a swap involves three separate copy operations, assuming no concurrent copying.
9. Items with indices less than or equal to `outer` are sorted.
10. c
11. d
12. copies
13. b
14. Items with indices less than `outer` are partially sorted.
15. b

Experiment 3-A: The bubble sort and insertion sort algorithms are stable. The selection sort is not stable. For example, the list `[('elm', 1), ('elm', 2), ('asp', 1)]` sorts to `[('asp', 1), ('elm', 2), ('elm', 1)]` when sorting by the first element.

Chapter 4, “Stacks and Queues”

1. 10
2. b
3. last-in, first-out and first-in, first-out
4. False

5. Neither; it doesn't move.
6. 45
7. False
8. c
9. $O(N)$
10. c
11. True
12. b
13. Yes. The `find()` method would need to be modified to find the lowest index into the array for the priority of the item being inserted (assuming high indices are at the front of the queue and highest priority). That's needed to maintain the FIFO ordering of the items with equal priority. The binary search algorithm in the `find()` method stops at the first index it finds with the matching priority key.
14. a (and maybe b if some keystrokes can have higher priority and need to be processed ahead of others in the queue)
15. a

Chapter 5, “Linked Lists”

1. c
2. first (called `__first` in `LinkedList`)
3. d
4. 2
5. 1 plus setting the `__next` field to `None` in the new link, which really is a nonreference.
6. b
7. `x.setNext(None)`

8. When the variable, x , is passed as a reference instead of a copy of its value
9. a
10. c
11. a linked list
12. once
13. a double-ended list
14. an ordered double-ended list
15. d
16. Usually, the list. They both do `push()` and `pop()` in $O(1)$ time, but the list uses memory more efficiently.
17. The array. Binary search allows $O(\log N)$ time to find the item by key, but if you use a list, even a doubly linked one, each search step has to walk through all the intermediate links to get to the next item to test.
18. b
19. Calling programs can change the link's next pointer to create or break circular lists, regardless of whether the list type supports them.
20. a

Chapter 6, "Recursion"

1. c
2. 101
3. d
4. 2. The base case returns 0, and `nth` has a value of 1 in the caller to the base case. That caller returns the first non-base case value of 1. The value of `nth` right after 1 is returned is in the call where `nth` has the value 2.
5. True

6. "ue"
7. c
8. moving a smaller number of disks
9. b
10. b
11. b
12. 11. There are calls to `mergesort()` for array sizes 1,024, 512, 256, 128, 64, 32, 16, 8, 4, 2, and 1 on the stack at one point.
13. stack
14. Both function arguments and local variables of a recursive function must be stored for each call that happens in the recursive execution. These can be stored in the problem descriptions and kept on a stack.
15. when the stack of problem descriptions is empty

Chapter 7, “Advanced Sorting”

1. c
2. 40
3. d
4. The complexity could degenerate to $O(N^2)$.
5. $O(N \times \log N)$, $O(N^2)$
6. a
7. pivot
8. d
9. True
10. c
11. partitioning the resulting subarrays recursively (to fully sort the array)
12. b

13. pivot
14. $\log_2 N$
15. For data that is initially forward or reverse sorted, the complexity becomes $O(N^2)$.
16. True
17. a
18. $O(N)$. It needs 10 linked lists and those lists must collectively store a full copy of the N elements.
19. d
20. b only, and it's a modified version of mergesort.

Chapter 8, “Binary Trees”

1. $O(\log N)$
2. c
3. False
4. b
5. 5
6. c
7. node, (binary search) tree
8. a
9. d
10. False
11. finding
12. b
13. linked list
14. A's key, the key of A's right child

- 15. d
- 16. d
- 17. $2 \times n + 2$
- 18. False
- 19. compress
- 20. c

Chapter 9, “2-3-4 Trees and External Storage”

- 1. b
- 2. lower in height; always balanced
- 3. 2
- 4. False
- 5. b
- 6. the root is split
- 7. a
- 8. 2
- 9. b
- 10. $O(\log N)$
- 11. d
- 12. at least one block's worth of
- 13. 2-3
- 14. a
- 15. c

Chapter 10, “AVL and Red-Black Trees”

- 1. in order (or inverse order) of the keys

2. b
3. c
4. c
5. a
6. b
7. False
8. d
9. b
10. rotations, changing the colors of nodes (swaps and flips)
11. red
12. left child, right child
13. d
14. a node (black), its two children (red)
15. b
16. True
17. a
18. d, Only one color swap is needed for a black node with two red children on insertion.
19. d
20. d

Chapter 11, “Hash Tables”

1. $O(1)$
2. hash function
3. the modulo operation
4. a collision

5. d
6. linear probing
7. (0), 1, 4, 9, 16, 25
8. b
9. a
10. linked list or binary search tree
11. 1.0
12. True
13. be a prime number
14. the array size
15. all of them; for the same key, an unsuccessful search must go through the whole probe sequence, whereas a successful search stops with the found item.
16. b
17. b
18. A sorted linked list is most efficient with 1 empty link for all N items. Next most efficient is the hash table at 0.6 load factor, which has $0.4 \times N$ empty or deleted cells. The AVL tree has two empty links for all its leaf nodes, which is up to $(N + 1)/2$, and one empty link for some of the parents of the leaf nodes. For a complete, balanced binary tree, there would be exactly $(N + 1)/2$ leaf nodes and no empty links among the internal nodes. Overall, the total would be close to $2 \times N/2$ or N empty cells. The actual count in a specific tree could vary from that, but it is not going to get below $N/2$, which is still more than $0.4 \times N$.
19. False
20. the same block

Chapter 12, “Spatial Data Structures”

1. The keys are a combination of two numbers, rather than a single integer or string.
2. Geographic or latitude-longitude
3. The distance function (and the separate structures for storing the points)
4. b
5. False
6. Only a portion of the spatial data is stored in each grid cell, reducing search time
7. Quadrees can adapt to clusters in spatial data while grids cells may end up having some cells very full and others be empty. They have $O(\log N)$ search time instead of $O(N)$ for grids and point lists.
8. d
9. a. $O(N)$ b. $O(N)$ c. $O(\log N)$
10. False
11. d
12. False
13. The quadrants are the northeast, northwest, southwest, and southeast regions around a node's center. Points lying on the boundary of two quadrants are placed in next quadrant in clockwise order.
14. d
15. False

Chapter 13, “Heaps”

1. b
2. Both the right and left children have keys less than (or equal to) the parent, but no ordering is specified between the child items.
3. root

4. d
5. a
6. c
7. array or linked list
8. sifting down because it must determine the maximum child key
9. b
10. one
11. $O(N \times \log N)$
12. slower because it takes more time per item but faster in some cases because it does not degenerate to $O(N^2)$
13. c
14. N
15. $N + K \times \log N$

Experiment 13-B: The heapsort algorithm is not stable. Consider an array of all equal keys such as 2, 2, 2, 2, 2, 2. It already meets the heap condition, so heapify doesn't reorder any items. The max (root) item is removed from the heap and goes to the last position of the array. The last of the remaining equal keyed items in the heap is then moved to the root and sifted down. Because all keys in the heap are equal, the item does not sift down and becomes the next item moved to the end. This repeats for all the equal keyed items, swapping the last for the first and then swapping them back again for each removal. Using the original indices of the six-element array—0, 1, 2, 3, 4, and 5 to track the item order—the final arrangement will be 1, 2, 3, 4, 5, 0.

Chapter 14, “Graphs”

1. edge, vertices (or sometimes, nodes)
2. Count the number of 1s or edges and divide by 2 (assuming the identity diagonal is all 0s).

3. vertex, edge
4. d
5. A:B, B:A $\rightarrow$ C $\rightarrow$ D, C:B, D:B $\rightarrow$ C
6. a
7. Three. If the vertices are A, B, and C, the minimum spanning trees are defined by the paths ABC, BCA, and CAB.
8. Option b, depth-first search, is the fastest because it takes $O(N^2)$ time. Option a first finds an MST using depth-first traversal over the entire connected component and then takes more time to find whether Z is in the MST. Option c runs in $O(N^2)$ time but doesn't work in all directed graphs because a breadth-first search would find only A and Z in the same component if there was a cycle including A and Z. Option d works but takes $O(N^3)$ time to compute the connectivity matrix.
9. c
10. directed acyclic graph (DAG) or tree (directed or undirected)
11. b
12. No
13. True
14. d
15. Yes. The adjacency matrix is asymmetric, so the graph is undirected. The result would be an exception because the graph contains a cycle, ABDA.
16. directed acyclic graph
17. b
18. Only if the goal is to find vertices with no edges. Every vertex with a undirected edge has a predecessor. The result would find all the vertices with no edges with the rest essentially forming a cycle (if cycles like ABA are allowed for a undirected edge AB).
19. Using the improved topological sort algorithm, computing time is $O(N^2)$. Because the first graph has 10 times as many vertices, the ratio

would be $10^2 = 100$. The basic topological search took $O(N^3)$, so its ratio would be $10^3 = 1,000$.

20. None of the algorithms mentioned in this chapter solve the bridges of Königsberg problem.

Chapter 15, “Weighted Graphs”

1. distances, costs, times
2. c
3. d
4. False (for undirected graphs)
5. the lowest-weight edge
6. b
7. False
8. is already in the MST
9. False
10. a
11. True
12. a
13. d
14. In Warshall’s algorithm, connectivity matrix cells are updated using bitwise-OR rules, whereas in the Floyd-Warshall algorithm, the cells are the updated using the minimum of their current value and that of adding two other cell values.
15. intractable
16. adjacency list, adjacency matrix
17. Floyd-Warshall
18. 2^N , where N is $K^2 - 1$, the number of squares on the board minus 1

19. No. The route mentioned in the “Traveling Salesperson Problem” section of the text had total cost of 140, whereas the proposed route costs 145. Neither is the minimum solution of the TSP.
20. when the cost (weight) of traveling in one direction between two vertices is not the same as traveling in the opposite direction